



von KARMAN INSTITUTE
FOR FLUID DYNAMICS



BRUFACE
BRUSSELS FACULTY
OF ENGINEERING



An Introduction to Variational Methods for Real Time Modeling

Miguel Alfonso Mendez

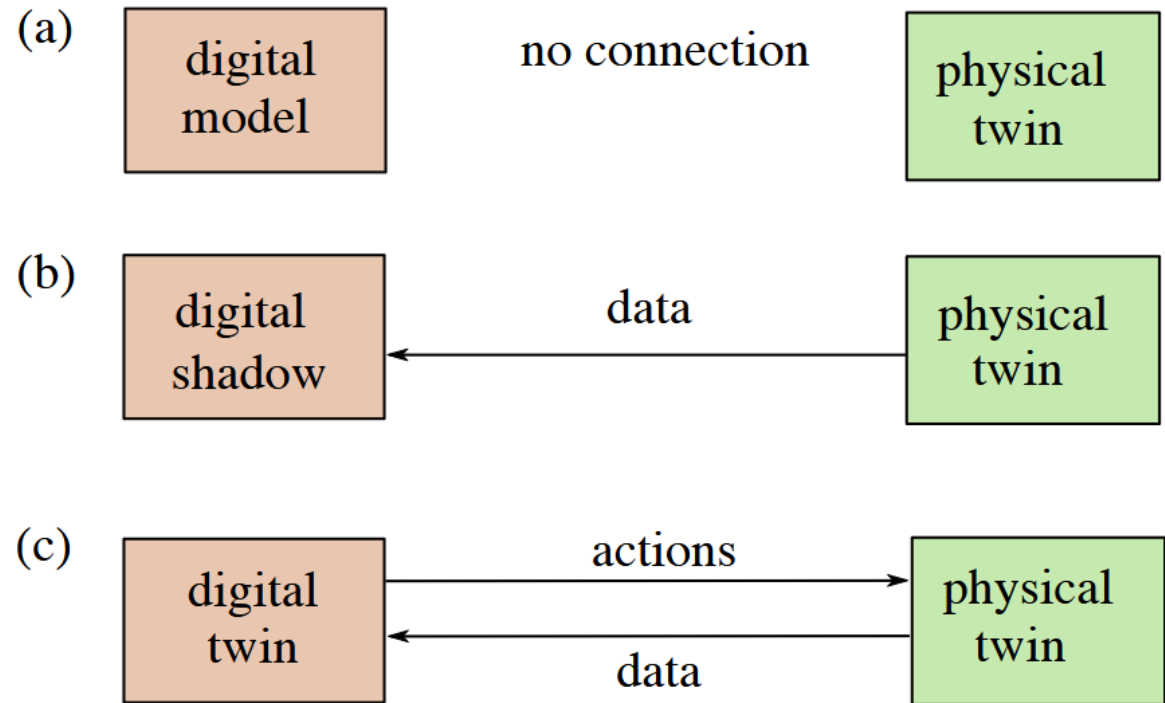
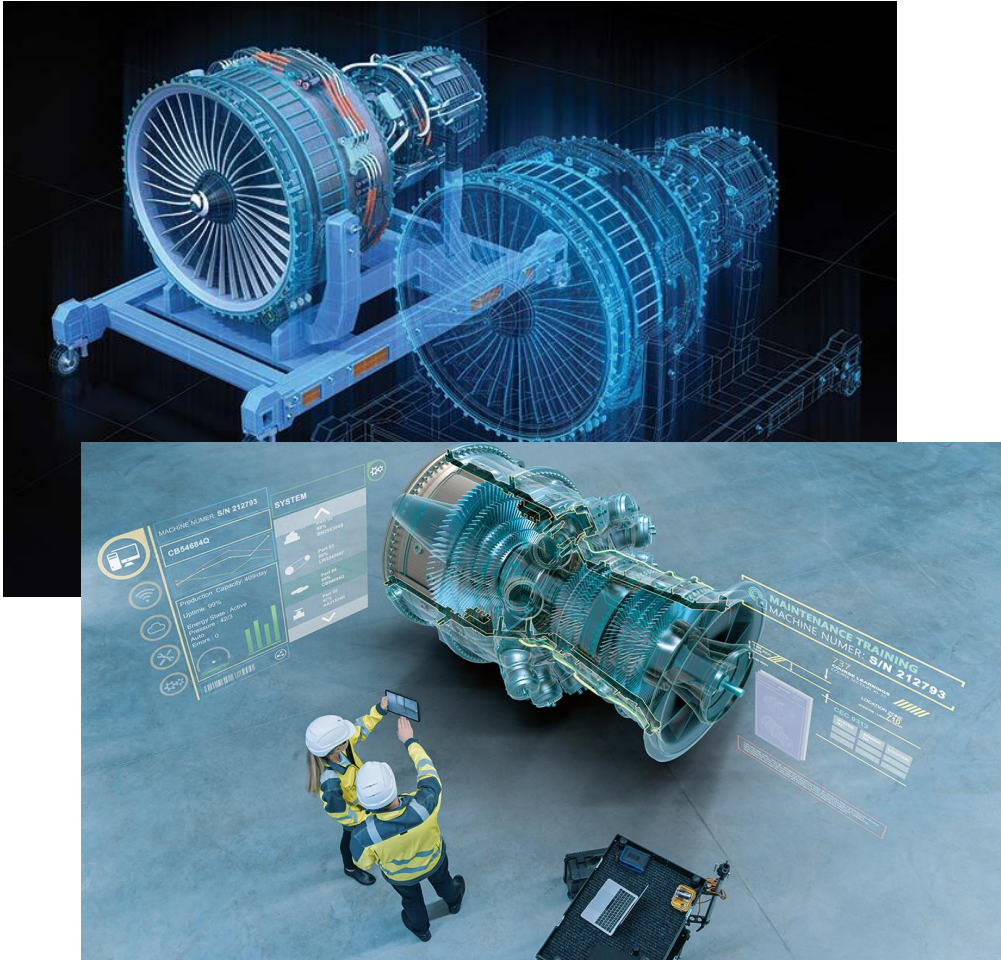
mendez@vki.ac.be

Table of Contents

1. A bit of context: real time modeling today... and digital twins !

What is a Digital Twin ? Adaptive modeling 2.0

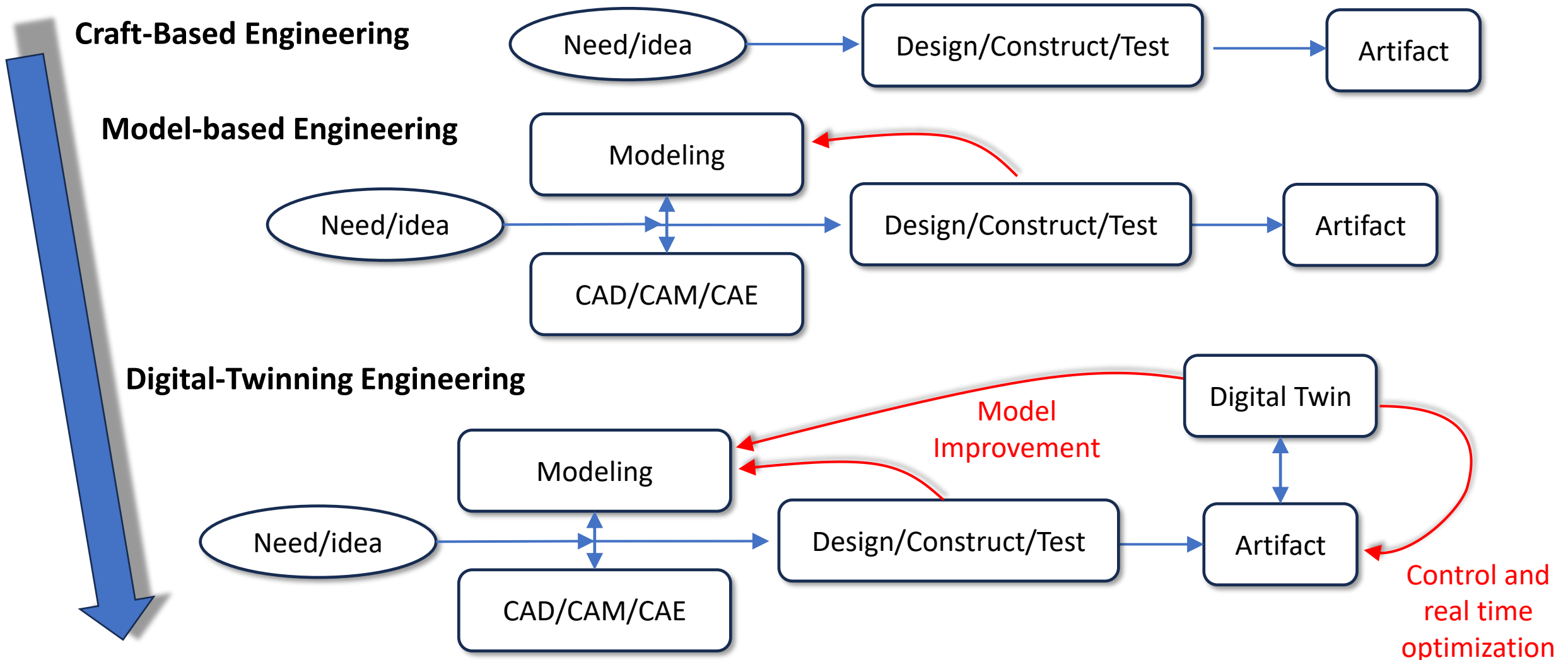
A digital twin is a time-evolving virtual representation (a model) of a physical system linked by a two-way data connection. It uses real-time sensor data to predict behavior and actively intervenes in the physical world to optimize performance, drive decision-making, and troubleshoot issues



The philosophical foundations of digital twinning, Wagg et al 2025

Perspectives: a change of paradigm?

Slide 4



The key enablers

1. Sensor technology: cost and availability

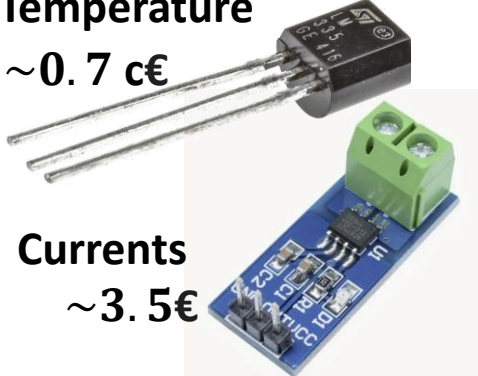
Light



~0.01 €

Temperature

~0.7 c€

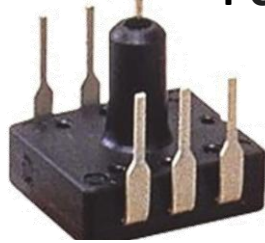


Currents

~3.5€

Pressure

~4 €



Acceleration

~6 €



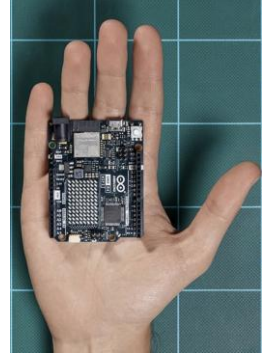
Camera (3MP)+ objective



~25 € + 20 €

Micro controller (wifi)

~30 €



2. Connectivity and Internet of Things



3. Machine Learning Engineering

Machine Learning + Domain knowledge + Data Assimilation and Inverse Modeling + Control Engineering

Table of Contents

1. A bit of context: real time modeling today... and digital twins !

2. [A General Template](#)

A general setting for modeling

Consider a dynamical system which depends on some parameters.
These can be link to the system's state by a closure law:

State vector and its time derivative

$$\mathbf{s}(t), \dot{\mathbf{s}}(t) \in \mathbb{R}^{n_s}$$

'engineering' model parameters

$$\mathbf{p}(t) \in \mathbb{R}^{n_p}$$

$$\begin{cases} \dot{\mathbf{s}}(t) &= \mathbf{f}(\mathbf{s}(t), \mathbf{p}) \\ \mathbf{p}(t) &= \tilde{\mathbf{g}}(\mathbf{s}(t); \mathbf{w}_p) \\ \mathbf{s}(0) &= \mathbf{s}_0 . \end{cases}$$

'surrogate' model parameters

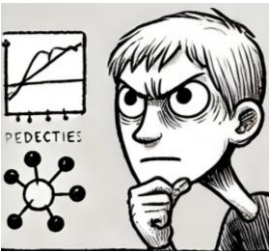
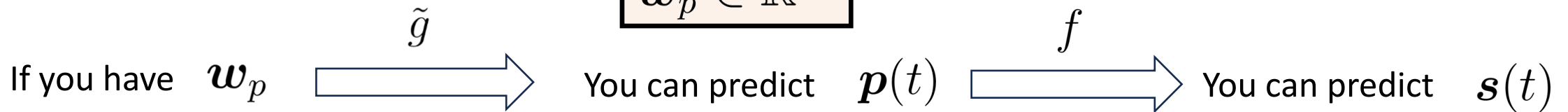
$$\mathbf{w}_p \in \mathbb{R}^{n_w}$$

(preferably from physics)

$$\mathbf{f} : \mathbb{R}^{n_s \times n_p} \rightarrow \mathbb{R}^{n_s}$$

closure law
(machine learning)

$$\tilde{\mathbf{g}} : \mathbb{R}^{n_s \times n_w} \rightarrow \mathbb{R}^{n_p}$$



Modelists

We look for the model parameters that minimize the discrepancy with the available data.

$$\mathcal{J}_p(\mathbf{w}_p)$$

A general setting for control

Consider a dynamical system which depends on some parameters.
These can be link to the system's state by a closure law:

State vector and its time derivative

$$\mathbf{s}(t), \dot{\mathbf{s}}(t) \in \mathbb{R}^{n_s}$$

Control Action

$$\mathbf{a}(t) \in \mathbb{R}^{n_a}$$

$$\begin{cases} \dot{\mathbf{s}}(t) &= f(\mathbf{s}(t), \mathbf{a}) \\ \mathbf{a}(t) &= \pi(\mathbf{s}(t); \mathbf{w}_a) \\ \mathbf{s}(0) &= \mathbf{s}_0 . \end{cases}$$

Policy Parameters

$$\mathbf{w}_a \in \mathbb{R}^{n_\pi}$$

(preferably from physics)

$$f : \mathbb{R}^{n_s \times n_p} \rightarrow \mathbb{R}^{n_s}$$

Action policy

$$\pi : \mathbb{R}^{n_s \times n_\pi} \rightarrow \mathbb{R}^{n_a}$$



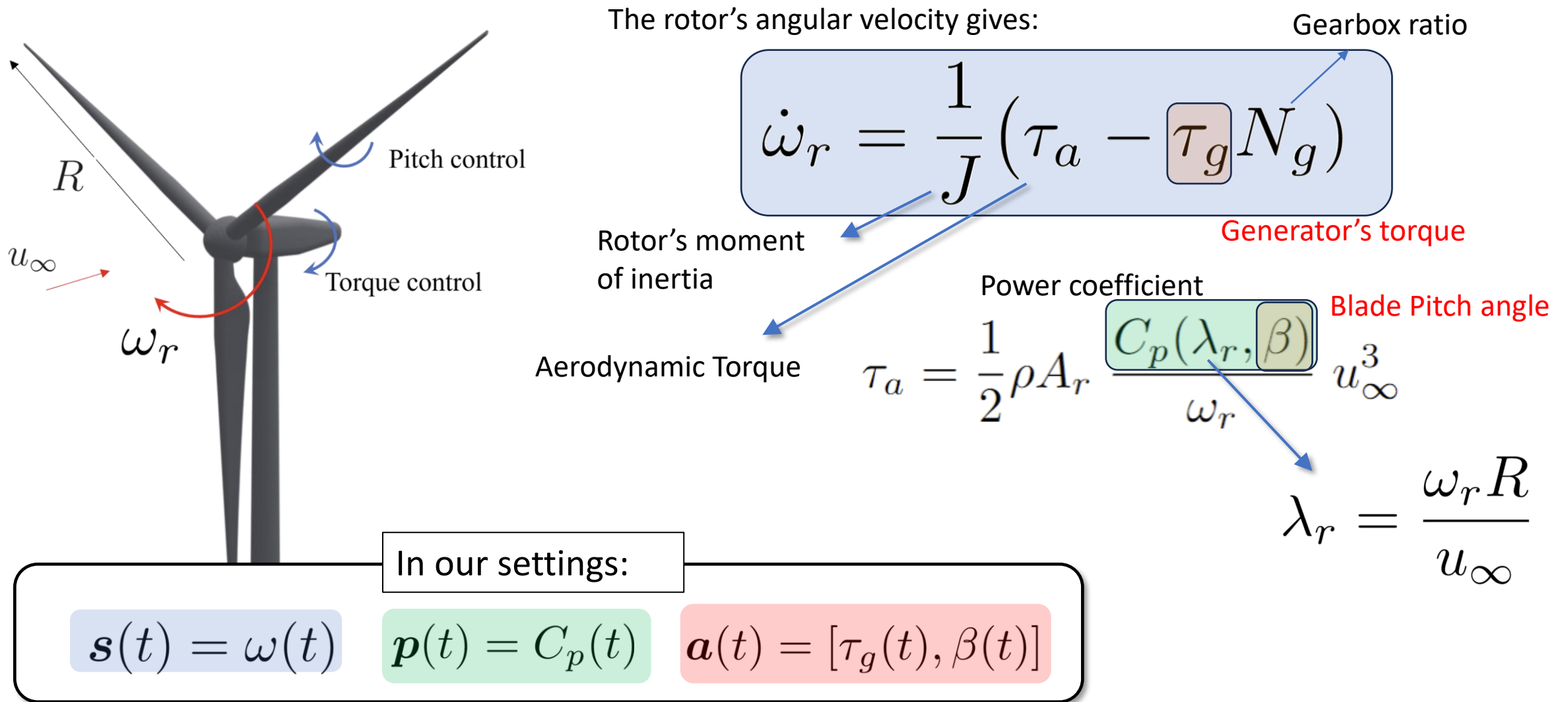
$$\mathcal{J}_a(\mathbf{w}_a)$$

We look for the policy parameters that drive the systems to the desired trajectory

Controlists



Preview for the final tutorial (spoiler!)



Control goal: maximize energy production without overloading the blades

Table of Contents

1. A bit of context: real time modeling today... and digital twins !

2. Possible challenges and tools

3. Review of some fundamental tools

3.1 Computing derivatives via Forward and Backward Methods

3.2 Stochastic and Momentum Accelerated Optimizers

Table of Contents

1. A bit of context: real time modeling today... and digital twins !

2. Possible challenges and tools

3. Review of some fundamental tools

3.1 Computing derivatives via Forward and Backward Methods

3.2 Stochastic and Momentum Accelerated Optimizers

The key definition

Given a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ linking two vectors

$$\mathbf{y} = f(\mathbf{x})$$

$$\mathbf{x} \in \mathbb{R}^n$$
$$\mathbf{y} \in \mathbb{R}^m$$

We define the first order derivatives (Jacobian) as

$$\frac{d\mathbf{y}}{d\mathbf{x}} = \begin{pmatrix} \partial_{x_1} y_1 & \partial_{x_2} y_1 & \cdots & \partial_{x_n} y_1 \\ \partial_{x_1} y_2 & \partial_{x_2} y_2 & \cdots & \partial_{x_n} y_2 \\ \vdots & \vdots & \ddots & \vdots \\ \partial_{x_1} y_m & \partial_{x_2} y_m & \cdots & \partial_{x_n} y_m \end{pmatrix} \in \mathbb{R}^{m \times n}$$

i.e.

$$\left. \frac{d\mathbf{y}}{d\mathbf{x}} \right|_{i,j} = \partial_{x_j} y_i$$

If $m=1$, this is the gradient of a scalar function. **Note that it is here a row vector.**

If $n=1$, we have a column vector

If $m=n=1$, this is the first derivative of a single variable function.

The key properties

We need three rules for differentiation:

1. Product Rule $\frac{d}{d\mathbf{x}} (f(\mathbf{x})g(\mathbf{x})) = \frac{df}{d\mathbf{x}} g(\mathbf{x}) + f(\mathbf{x}) \frac{dg}{d\mathbf{x}}$

2. Sum Rule $\frac{d}{d\mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})) = \frac{df}{d\mathbf{x}} + \frac{dg}{d\mathbf{x}}$

3. Chain Rule $\frac{d}{d\mathbf{x}} (f \circ g)(\mathbf{x}) = \frac{d}{d\mathbf{x}} (g(f(\mathbf{x}))) = \frac{dg}{df} \frac{df}{d\mathbf{x}}$

Care must be taken with the size of the matrices/vectors involved.

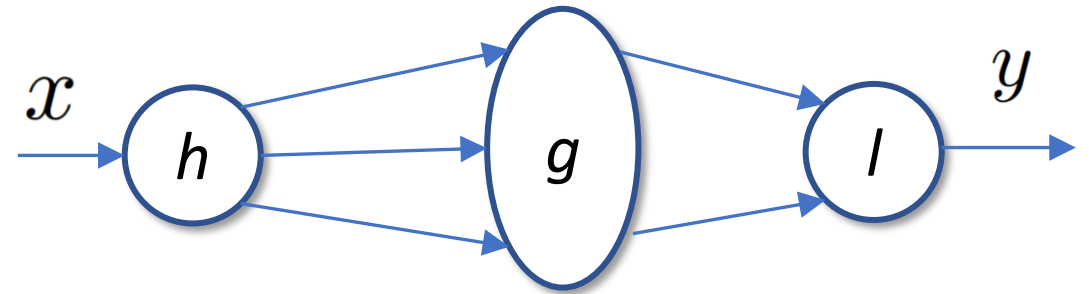
A computational graph is usually helpful. For example, consider the nested functions:

$$y = (l \circ g \circ h)(x)$$

with:

$$\begin{aligned} l &: \mathbb{R}^2 \rightarrow \mathbb{R} \\ g &: \mathbb{R}^3 \rightarrow \mathbb{R}^2 \\ h &: \mathbb{R} \rightarrow \mathbb{R}^3 \end{aligned}$$

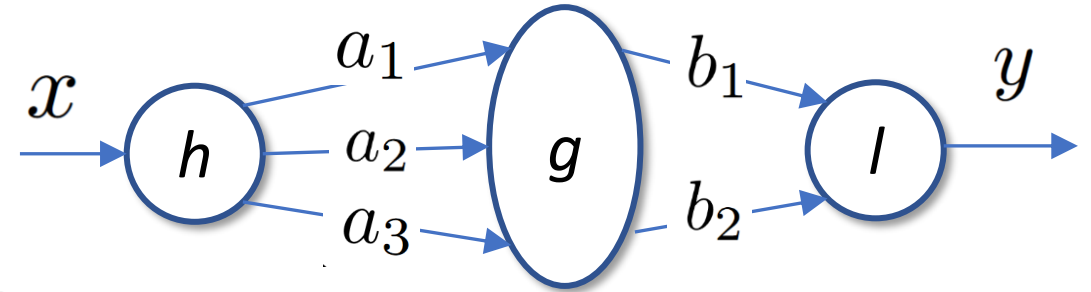
the graph is:



Practicing with the Chain Rule

Practice! The following functions have the same computational graph previously shown

$$y = (l \circ g \circ h)(x)$$



$$\mathbf{a} = h(x) = \begin{cases} a_1 &= x^2 \\ a_2 &= x^3 - 1 \\ a_3 &= 3x \end{cases} \quad \mathbf{b} = g(\mathbf{a}) = \begin{cases} b_1 &= a_1 a_2 \\ b_2 &= a_1 a_3 \end{cases} \quad y = l(\mathbf{b}) = b_1^2 + b_2^2$$

$(1 \times 2) \quad (3 \times 1)$

What is the derivative $\frac{dy}{dx} =$

$(1 \times 1) \quad (2 \times 3)$

You should get: $\frac{dy}{dx} = \begin{bmatrix} 2b_1 & 2b_2 \end{bmatrix} \begin{bmatrix} a_2 & a_1 & 0 \\ a_3 & 0 & a_1 \end{bmatrix} \begin{bmatrix} 2x \\ 3x^2 \\ 3 \end{bmatrix}$

Fundamentals of Automatic Differentiation

Note that the evaluation of each term requires the evaluation of the intermediate functions also. Therefore, the order in which we compute it matters. We have two options:

Forward mode differentiation

(from input to output):

$$(x), (\mathbf{a}, \frac{d\mathbf{a}}{dx}), (\mathbf{b}, \frac{d\mathbf{b}}{d\mathbf{a}}), (y, \frac{dy}{d\mathbf{b}})$$

$$\frac{dy}{dx} = \frac{dy}{d\mathbf{b}} \left(\frac{d\mathbf{b}}{d\mathbf{a}} \frac{d\mathbf{a}}{dx} \right)$$

Reverse mode differentiation

(from output to input):

$$(y, \frac{dy}{d\mathbf{b}}), (\mathbf{b}, \frac{d\mathbf{b}}{d\mathbf{a}}), (\mathbf{a}, \frac{d\mathbf{a}}{dx})$$

$$\frac{dy}{dx} = \left(\frac{dy}{d\mathbf{b}} \frac{d\mathbf{b}}{d\mathbf{a}} \right) \frac{d\mathbf{a}}{dx}$$

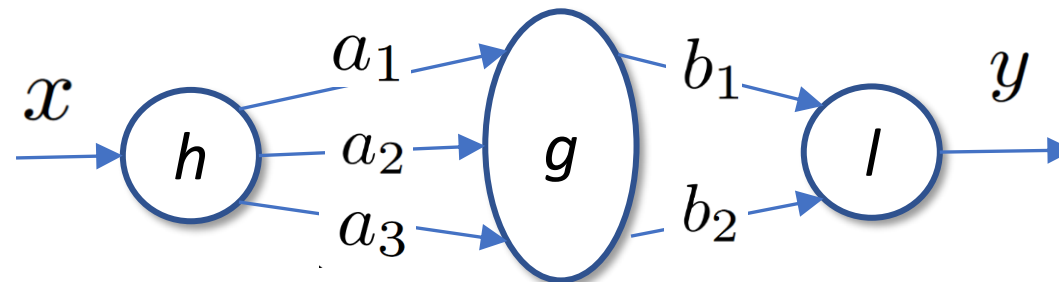


Table of Contents

1. A bit of context: real time modeling today... and digital twins !

2. Possible challenges and tools

3. Review of some fundamental tools

3.1 Computing derivatives via Forward and Backward Methods

3.2 Stochastic and Momentum Accelerated Optimizers

Common types of gradient descent

All gradient descent methods rely on the same principle:

$$\mathbf{w}^{i+1} = \mathbf{w}^i - \overset{\text{Learning Rate}}{\eta} \nabla_{\mathbf{w}} J(\mathbf{w}) \quad \eta \in \mathbb{R}$$

Based on the amount of data used for the gradient we have three families:

Batch Gradient Descent (BGD)

Gradient on the entire dataset: $\nabla_{\mathbf{w}} J(\mathbf{w}) = \nabla_{\mathbf{w}} J(\mathbf{w}, \mathbf{x}, \mathbf{y})$

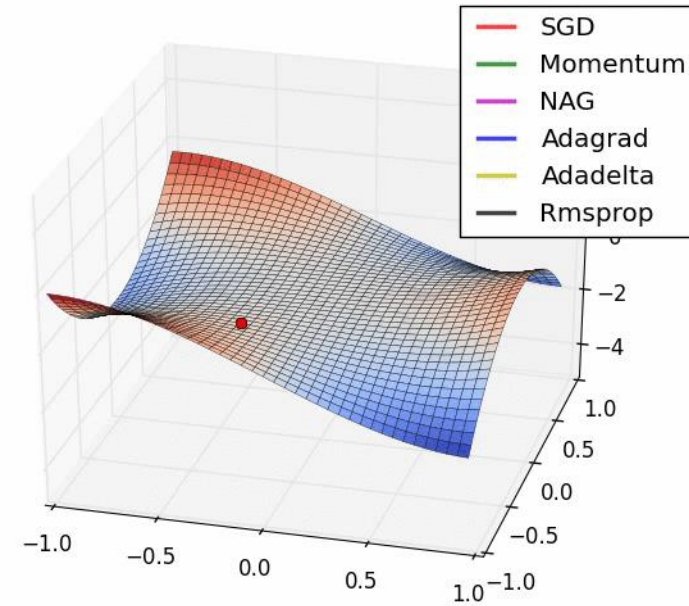
Stochastic Gradient Descent (SGD)

Gradient on each point: $\nabla_{\mathbf{w}} J(\mathbf{w}) = \nabla_{\mathbf{w}} J(\mathbf{w}, x_i, y_i)$

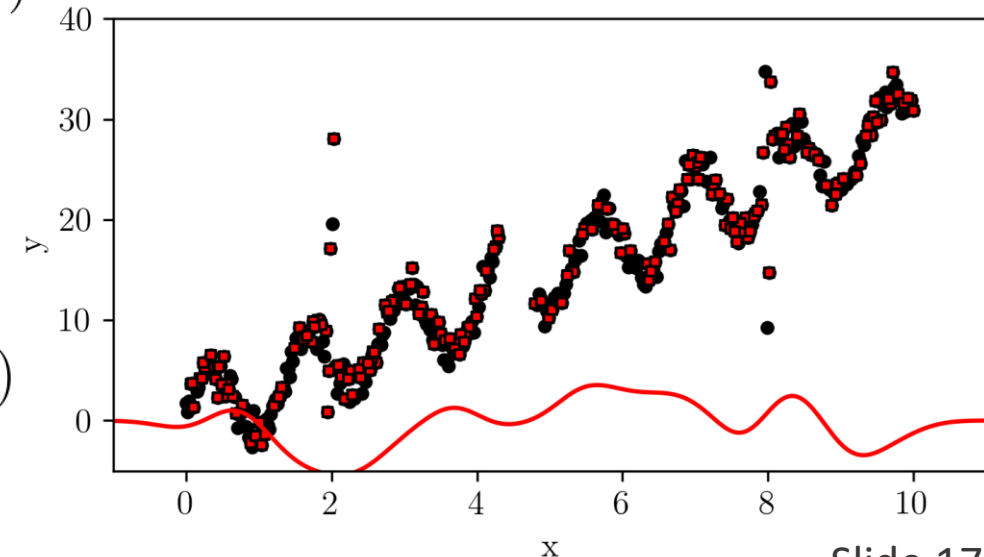
Mini-Batch Gradient Descent (MGD)

Gradient on N points: $\nabla_{\mathbf{w}} J(\mathbf{w}) = \nabla_{\mathbf{w}} J(\mathbf{w}, x_{i:i+N}, y_{i:i+N})$

Popular variants include momentum and/or gradient rescaling.



Animation by Alec Radford



Some classic ANN optimizers

$$\nabla_{\mathbf{w}} J(\mathbf{w} + \beta \mathbf{m}) \quad \text{Nesterov Acceleration}$$

$$\mathbf{w}^{i+1} = \mathbf{w}^i + \mathbf{m}^i$$

Basic Momentum

$$\mathbf{m}^{i+1} = \beta \mathbf{m}^i - \eta \nabla_{\mathbf{w}} J(\mathbf{w}) \quad \eta, \beta \in \mathbb{R}; \mathbf{m} \in \mathbb{R}^{n_p}$$

RMS Prop

$$\mathbf{w}^{i+1} = \mathbf{w}^i - \eta \nabla_{\mathbf{w}} J(\mathbf{w}) \oslash \sqrt{s^i + \varepsilon}$$

$$s^i = \beta s^{i-1} + (1 - \beta) \nabla_{\mathbf{w}} J(\mathbf{w}) \otimes \nabla_{\mathbf{w}} J(\mathbf{w}) \quad \eta, \beta, \varepsilon \in \mathbb{R}$$

$$\mathbf{w}^{i+1} = \mathbf{w}^i - \eta \hat{\mathbf{m}}^i \oslash \sqrt{\hat{\mathbf{s}}^i + \varepsilon}$$

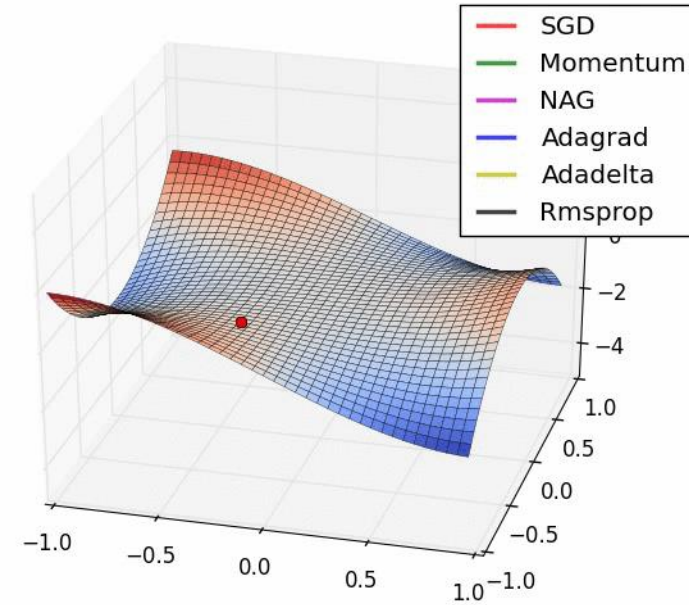
ADaptive Momentum

$$\mathbf{s}^{i+1} = \beta \mathbf{s}^i + (1 - \beta) \nabla_{\mathbf{w}} J(\mathbf{w}) \otimes \nabla_{\mathbf{w}} J(\mathbf{w})$$

Generally, ADAM is a good default choice!

$$\mathbf{m}^{i+1} = \beta_1 \mathbf{m}^i + (1 - \beta_1) \nabla_{\mathbf{w}} J(\mathbf{w})$$

$$\hat{\mathbf{m}} = \frac{\mathbf{m}}{1 - \beta_1^i} \quad \hat{\mathbf{s}} = \frac{\mathbf{s}}{1 - \beta_2^i} \quad \eta, \beta, \varepsilon \in \mathbb{R}; \mathbf{s}, \mathbf{m} \in \mathbb{R}^{n_p}$$



Animation by Alec Radford

Table of Contents

1. A bit of context: real time modeling today... and digital twins !

2. Possible challenges and tools

3. Review of some fundamental tools

3.1 Computing derivatives via Forward and Backward Methods

3.2 Stochastic and Momentum Accelerated Optimizers

[4. Tutorial 1](#)



Check 1_Adam_Optimization_Demo_to_FILL

✓ Ex 1 Visualizing ADAM Optimization on a Simple Polynomial Cost Function

This notebook demonstrates how the ADAM optimizer performs on a simple 2D polynomial cost function with multiple local minima.

We'll define a cost function with several quadratic basins and show how ADAM, thanks to its momentum and adaptive step size, efficiently finds a minimum without getting stuck in local traps.

```
1 def m_and_grad_numpy(x, alpha=1.0):
2     x1, x2 = x # input
3     # z function and its Jacobian
4     # (see the computational graph in BB)
5     z = np.array([
6         x2,
7         3 * x1**2 - x2,
8         4 * x1 - x2**3
9     ])
10    dz_dx = np.array([
11        [],
12        [],
13        []
14    ])
```

Work on it !

Table of Contents

1. A bit of context: real time modeling today... and digital twins !

2. Possible challenges and tools

3. Review of some fundamental tools

3.1 Computing derivatives via Forward and Backward Methods

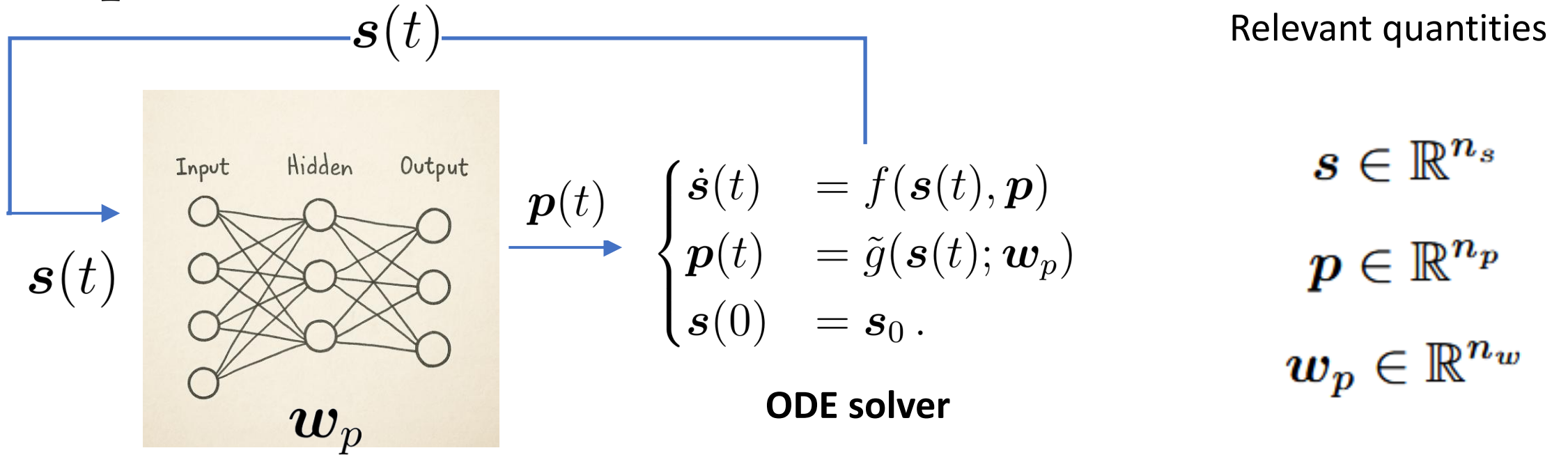
3.2 Stochastic and Momentum Accelerated Optimizers

4. Tutorial 1

[5. The next step](#)



The problem set



We want to minimize the following cost function (this is the simplest possible!)

$$\mathcal{J}(\mathbf{w}_p) = \frac{1}{T} \int_0^T \|\dot{\mathbf{s}}(t) - \mathbf{s}(\mathbf{p}(\mathbf{w}_p))\|^2 dt = \frac{1}{T} \int_0^T \mathcal{L}(\mathbf{s}(\mathbf{p}(\mathbf{w}_p))) dt$$

How to compute

$$\frac{d\mathcal{J}}{d\mathbf{w}_p} = \frac{1}{T} \int_0^T \frac{d\mathcal{L}}{d\mathbf{w}_p} dt \quad ?$$